

Task-1: Find the value of $\frac{dy}{dx}$ and $\frac{d^2y}{dx^2}$ at $x=1.2$ using Newton Forward Differentiation:

x	1.0	1.2	1.4	1.6	1.8	2.0	2.2
y=e ^x	2.7183	3.3201	4.0552	4.9530	6.0496	7.3891	9.0250

Code (Python):

```
from prettytable import PrettyTable
import math

value_x=[1.0,1.2,1.4,1.6,1.8,2.0,2.2]
value_y=[2.7183,3.3201,4.0552,4.9530,6.0496,7.3891,9.0250]
table = PrettyTable()

def create_table(n):
    field=["x","y","Δy"]
    for i in range(2,n+1):
        field.append(f'Δ{i}y')
    table.field_names=field
    table.float_format=".5"

def Differentiation(a,b):
    return (b-a)/1e-9

def Differentiation2(a,b,c):
    return (c-2*b+a)/1e-18

def Forward_difference(n):
    create_table(len(value_y)-1)
    del_value=[]
    temp=[]
    for i in range(1,len(value_y)):
        temp.append(value_y[i]-value_y[i-1])
    del_value.append(temp)

    for i in range(1,len(value_y)-1):
        temp=[]
        for j in range(1,len(del_value[i-1])):
            temp.append(del_value[i-1][j]-del_value[i-1][j-1])
        del_value.append(temp)

    for i in range(len(value_x)):
        row=[value_x[i],value_y[i]]
```

```

for j in range(len(del_value)):
    if i+1>len(del_value[j]):
        row.append("")
        continue
    row.append(del_value[j][i])
table.add_row(row=row)

h=abs(value_x[0]-value_x[1])
p=(n-value_x[0])/h
q=p
result1=0
result2=0
x_h=(p+1e-9)
x_2h=(p+2e-9)
for i in range(1,len(del_value)+1):
    dy_dp=Differentiation(q,x_h)
    dy_dp_2=Differentiation2(q,x_h,x_2h)
    result1+=del_value[i-1][0]*dy_dp/math.factorial(i)
    if i>1:
        result2+=del_value[i-1][0]*dy_dp_2/math.factorial(i)
    q*=(p-i)
    x_h*=(p-i+1e-9)
    x_2h*=(p-i+2e-9)

return (result1/h),(result2/h**2)
result=Forward_difference(1.2)
print(table)
print(f'y'(1.2)= {result[0]:.5f}\ny''(1.2)= {result[1]:.5f}')

```

Output:

x	y	Δy	$\Delta^2 y$	$\Delta^3 y$	$\Delta^4 y$	$\Delta^5 y$	$\Delta^6 y$
1.00000	2.71830	0.60180	0.13330	0.02940	0.00670	0.00130	0.00010
1.20000	3.32010	0.73510	0.16270	0.03610	0.00800	0.00140	
1.40000	4.05520	0.89780	0.19880	0.04410	0.00940		
1.60000	4.95300	1.09660	0.24290	0.05350			
1.80000	6.04960	1.33950	0.29640				
2.00000	7.38910	1.63590					
2.20000	9.02500						

$y'(1.2) = 3.32023$
 $y''(1.2) = 3.32107$

Task-2: Find the value of $\frac{dy}{dx}$ and $\frac{d^2y}{dx^2}$ at $x=2.2$ using Newton Backward Differentiation:

x	1.0	1.2	1.4	1.6	1.8	2.0	2.2
$y=e^x$	2.7183	3.3201	4.0552	4.9530	6.0496	7.3891	9.0250

Code (Python):

```
from prettytable import PrettyTable
import math

value_x=[1.0,1.2,1.4,1.6,1.8,2.0,2.2]
value_y=[2.7183,3.3201,4.0552,4.9530,6.0496,7.3891,9.0250]
table = PrettyTable()

def create_table(n):
    field=["x","y","Δy"]
    for i in range(2,n+1):
        field.append(f'Δ{i}y')
    table.field_names=field
    table.float_format=".5"

def Differentiation(a,b):
    return (b-a)/1e-9

def Differentiation2(a,b,c):
    return (c-2*b+a)/(1e-18)

def Forward_difference(n):
    create_table(len(value_y)-1)
    del_value=[]
    temp=[]
    for i in range(1,len(value_y)):
        temp.append(value_y[i]-value_y[i-1])
    del_value.append(temp)

    for i in range(1,len(value_y)-1):
        temp=[]
        for j in range(1,len(del_value[i-1])):
            temp.append(del_value[i-1][j]-del_value[i-1][j-1])
        del_value.append(temp)

    for i in range(len(value_x)):
        row=[value_x[i],value_y[i]]
```

```

for j in range(len(del_value)):
    if i+1>len(del_value[j]):
        row.append("")
        continue
    row.append(del_value[j][i])
table.add_row(row=row)

h=abs(value_x[0]-value_x[1])
p=(n-value_x[-1])/h
q=p
result1=0
result2=0
x_h=(p+1e-9)
x_2h=(p+2e-9)
for i in range(1,len(del_value)+1):
    dy_dp=Differentiation(q,x_h)
    dy_dp_2=Differentiation2(q,x_h,x_2h)
    result1+=del_value[i-1][-1]*dy_dp/math.factorial(i)
    if i>1:
        result2+=del_value[i-1][-1]*dy_dp_2/math.factorial(i)
    q*=(p+i)
    x_h*=(p+i+1e-9)
    x_2h*=(p+i+2e-9)

return (result1/h),(result2/h**2)

result=Forward_difference(2.2)
print(table)
print(f'y'(2.2)={result[0]:.5f}\ny"(2.2)={result[1]:.5f}')

```

Output:

x	y	Δy	$\Delta^2 y$	$\Delta^3 y$	$\Delta^4 y$	$\Delta^5 y$	$\Delta^6 y$
1.00000	2.71830	0.60180	0.13330	0.02940	0.00670	0.00130	0.00010
1.20000	3.32010	0.73510	0.16270	0.03610	0.00800	0.00140	
1.40000	4.05520	0.89780	0.19880	0.04410	0.00940		
1.60000	4.95300	1.09660	0.24290	0.05350			
1.80000	6.04960	1.33950	0.29640				
2.00000	7.38910	1.63590					
2.20000	9.02500						

$y'(2.2)=9.02290$
 $y''(2.2)=8.99398$

Task-3: Find the value of $\int_0^1 \frac{1}{1+x} dx$ using Trapezoid, Simpson 1/3 rule, Simpson 3/8 rule:

Code (Python):

```
# Integration for y=1/(1+x) function using Trapezoid rule

def func_value(x):
    return 1/(1+x)

def Trapezoid(h,x_0,x_n):
    n=int((x_n-x_0)/h)

    result=func_value(x_0)+func_value(x_n)
    for i in range(1,n):
        result+=2*(func_value(x_0+i*h))

    return result*(h/2)

result1=Trapezoid(0.5,0,1)
result2=Trapezoid(0.25,0,1)
result3=Trapezoid(0.125,0,1)
print(f'For h=0.5, I= {result1:.5f} from x= 0 to 1")
print(f'For h=0.25, I= {result2:.5f} from x= 0 to 1")
print(f'For h=0.125, I= {result3:.5f} from x= 0 to 1")
```

Output:

```
For h=0.5, I= 0.70833 from x= 0 to 1
For h=0.25, I= 0.69702 from x= 0 to 1
For h=0.125, I= 0.69412 from x= 0 to 1
```

Code (Python):

```
# Integration for y=1/(1+x) function using Simpson 1/3 rule

def func_value(x):
    return 1/(1+x)

def Simpson_1_by_3_rule(h,x_0,x_n):
    n=int((x_n-x_0)/h)

    result=func_value(x_0)+func_value(x_n)
    for i in range(1,n):
        if i%2==0:
            result+=2*(func_value(x_0+i*h))
```

```

        else:
            result+=4*(func_value(x_0+i*h))

    return result*(h/3)

result1=Simpson_1_by_3_rule(0.5,0,1)
result2=Simpson_1_by_3_rule(0.25,0,1)
result3=Simpson_1_by_3_rule(0.125,0,1)
print(f'For h=0.5, I= {result1:.5f} from x= 0 to 1")
print(f'For h=0.25, I= {result2:.5f} from x= 0 to 1")
print(f'For h=0.125, I= {result3:.5f} from x= 0 to 1")

```

Output:

```

For h=0.5, I= 0.69444 from x= 0 to 1
For h=0.25, I= 0.69325 from x= 0 to 1
For h=0.125, I= 0.69315 from x= 0 to 1

```

Code (Python):

```

# Integration for y=1/(1+x) function using Simpson 3/8 rule

def func_value(x):
    return 1/(1+x)

def Simpson_3_by_8_rule(n,x_0,x_n):
    h=(x_n-x_0)/n

    result=func_value(x_0)+func_value(x_n)
    for i in range(1,n):
        if i%3==0:
            result+=2*(func_value(x_0+i*h))
        else:
            result+=3*(func_value(x_0+i*h))

    return result*(3*h/8)

result1=Simpson_3_by_8_rule(5,0,1)
result2=Simpson_3_by_8_rule(10,0,1)
result3=Simpson_3_by_8_rule(15,0,1)
print(f'For n=5, I= {result1:.5f} from x= 0 to 1")
print(f'For n=10, I= {result2:.5f} from x= 0 to 1")
print(f'For n=15, I= {result3:.5f} from x= 0 to 1")

```

Output:

```
For n=5, I= 0.67946 from x= 0 to 1  
For n=10, I= 0.68035 from x= 0 to 1  
For n=15, I= 0.69315 from x= 0 to 1
```

Discussion:

Here I have discussed the newton's forward and backward differentiation and then trapezoidal rule, Simpson 1/3 rule and Simpson 3/8 rule.

In forward differentiation we must differentiate the newton's forward difference with respect to x and same for backward differentiation. After generating the Δy value, put them in the equation and calculate the differentiation.

After then integration, we must integrate the newton's forward difference with respect to x . Then set $n=1$ for trapezoidal rule, $n=2$ for Simpson's 1/3 rule, $n=3$ for Simpson's 3/8 rule and calculate the integral system.